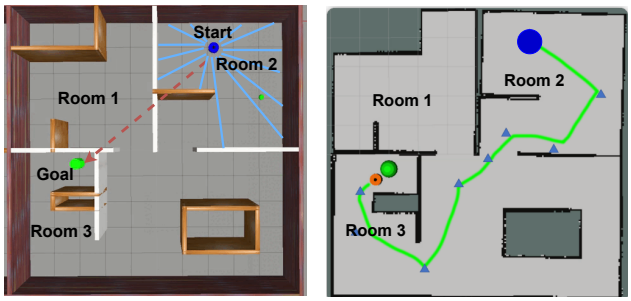
2025 IEEE International Conference on Robotics and Automation (ICRA)
May 19-23, 2025. Atlanta, USA

Hierarchical Reinforcement Learning for Safe Mapless Navigation with Congestion Estimation

Jianqi Gao[1], Xizheng Pang[1], Qi Liu[2] and Yanjie Li[1]

***Abstract*— Reinforcement learning-based mapless navigation holds significant potential. However, it faces challenges in indoor environments with local minima area. This paper introduces a safe mapless navigation framework utilizing hierarchical reinforcement learning (HRL) to enhance navigation through such areas. The high-level policy creates a sub-goal to direct the navigation process. Notably, we have developed a sub-goal update mechanism that considers environment congestion, efficiently avoiding the entrapment of the robot in local minimum areas. The low-level motion planning policy, trained through safe reinforcement learning, outputs real-time control instructions based on acquired sub-goal. Specifically, to enhance the robot's environmental perception, we introduce a new obstacle encoding method that evaluates the impact of obstacles on the robot's motion planning. To validate the performance of our HRL-based navigation framework, we conduct simulations in office, home, and restaurant environments. The findings demonstrate that our HRL-based navigation framework excels in both static and dynamic scenarios. Finally, we implement the HRL-based navigation framework on a TurtleBot3 robot for validation experiments, which exhibits its strong generalization capabilities.**



(a) Environment (b) Navigation path with sub-goal

Fig. 1. An illustration of HRL-based safe mapless navigation. In (a), the blue dot in room 2 marks the robot's start position, while the green dot in room 3 indicates the pre-specified goal position. The red dotted arrow shows the direction from start to goal, and the blue line is the laser scan from the lidar. Lacking a global map, the robot relies on relative goal information and may follow the red arrow, potentially trapping it in room 2. In (b), the green curve is the navigation path with the blue triangle as a sub-goal, and the red dot marks the robot's current position.

I. Introduction

Navigation is among the most fundamental functionalities of robots [1]. While traditional navigation algorithms are adept at ensuring that robots safely and efficiently accomplish their designated tasks, they often encounter difficulties when adapting to environments that are mapless, dynamic and unstructured [2]. Although mapless navigation algorithms, such as the Bug algorithm [3], mitigate some of these challenges, they tend to produce excessively long paths

In recent years, researchers have acknowledged that reinforcement learning provides robots with the ability to autonomously learn and adapt in unfamiliar environments, offering significant advantages in generalization and real-time decision-making [4]. However, reinforcement learning-based mapless navigation faces challenges such as robots frequently encountering local minimum areas [5] that hinder task completion, a tendency to take detours [6] due to the low likelihood of selecting direct actions from the neural network's output, and the lack of effective safety guarantees, potentially leading to collisions [7].

As depicted in Fig. 1, this paper introduces a mapless navigation framework leveraging hierarchical reinforcement learning (HRL) to address the aforementioned challenges. The high-level policy formulates sub-goals that direct the robot's navigation. In particular, different from existing sub-goal generation policies [8]–[11], we have developed a sub-goal update mechanism that accounts for environment congestion, effectively avoiding local minima areas. Concurrently, the low-level motion planning policy, honed through safe reinforcement learning (SRL), produces real-time control directives utilizing the achieved sub-goal, lidar-derived obstacle data, and current robot status. To enhance the robot's environmental awareness, we've implemented an innovative obstacle encoding technique that quantifies obstacles' effects on motion planning. To evaluate the navigation capabilities of the proposed HRL-based framework, we simulate indoor environments, including office, home, and restaurant, within Gazebo and performe physical tests using the TurtleBot3-Burger robot. The results demonstrate that the HRL-based framework can navigate through local minimum areas.

The remainder of this paper is structured as follows. Section II presents the problem formulation. Section III introduces the HRL-based navigation framework. Section IV presents the simulation experiment results. Section V shows the physical robots validation. Section VI provides the conclusion.

This work was supported in part by the Shenzhen Fundamental Research Program under Grant JCYJ20220818102415033, JSGG20201103093802006 and KJZD20230923114222045 and in part by the National Natural Science Foundation of China under Grant 61977019 and U1813206. (*Jianqi Gao and Xizheng Pang contribute equally. Corresponding author: Yanjie Li and Qi Liu.*)

[1]Jianqi Gao, Xizheng Pang, and Yanjie Li are with the Department of Guangdong Key Laboratory of Intelligent Morphing Mechanisms and Adaptive Robotics and School of Inteligence Science and Engineering, Harbin Institute of Technology, Shenzhen, 518055, China. gaojianqi205a@stu.hit.edu.cn, xzpang@stu.hit.edu.cn, autolyj@hit.edu.cn

[2]Qi Liu is with the Faculty of Robot Science and Engineering, Northeastern University, 110819, Shenyang, China. liuqi@mail.neu.edu.cn

2025 IEEE International Conference on Robotics and Automation (ICRA) | 979-8-3315-4139-2/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICRA55743.2025.11128867

Authorized licensed use limited to: HUNAN UNIVERSITY. Downloaded on October 21,2026 at 09:12:10 UTC from IEEE Xplore. Restrictions apply.

II. Problem Formulation

To simplify and formalize the description, the robot's navigation can be framed as a finite-horizon Partially Observable Markov Decision Process (POMDP), denoted by the tuple $\langle h, S, A, \Omega, T, O, R, \gamma, b_0 \rangle$. Here, h represents the planning horizon, while S, A, and Ω denote the sets of states, actions, and observations, respectively. The transition model T is stochastic, with $T(s^t, a^t, s^{t+1}) = P(s^{t+1} \mid s^t, a^t)$ indicating the probability of transitioning to state s^{t+1} from state s^t after taking action a^t. The observation model O is given by $O(o^{t+1}, s^{t+1}, a^t) = P(o^{t+1} \mid s^{t+1}, a^t)$, representing the likelihood of observing o^{t+1} when reaching state s^{t+1} following action a^t. The reward function R is bounded, where $R(s^t, a^t)$ is the reward received for performing action a^t in state s^t. The discount factor γ, ranging from 0 to 1, balances the importance of immediate versus future rewards. Lastly, b_0 is the initial state distribution, with $b_0(s^t)$ equal to the probability of the system's initial state being s^t.

As depicted in Fig. 2, we address the POMDP using a hierarchical framework. The high-level policy generates sub-goals, while the low-level policy, conditioned on these sub-goals, produces executable actions. The high-level policy activates when the robot is within a distance d_u^t (defined in (5)) from the current sub-goal. With a given sub-goal, the low-level policy determines actions at each time step to guide the robot towards the new sub-goal while evading both static and dynamic obstacles. Formally, the low-level sub-goal-conditioned policy is denoted by $\pi(a^t \mid s^t, g_{s,i})$, where $g_{s,i}$ is the new ith sub-goal. The low-level policy essentially solves a short-horizon goal-reaching POMDP defined between two consecutive high-level sub-goals.

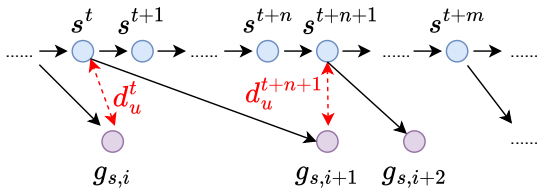


Fig. 2. The workflow of the proposed HRL-based navigation framework.

III. HRL-Based Navigation Framework

A. Overview

The proposed HRL-based navigation framework is depicted in Fig. 3. The high-level policy uses environment data to generate sub-goals within the robot's lidar range, guiding motion planning. The low-level policy issues real-time control commands based on environment, robot status, and sub-goal data, allowing for agile obstacle and pedestrian avoidance. Due to their independence, both policies can be trained and deployed separately.

B. High-Level Sub-Goal Policy

1) Observation Space: The high-level policy network's observation space integrates lidar point cloud data $\mathbb{L}^t = [l_1^t, l_2^t, l_3^t, \ldots, l_n^t]$ (n is 1080, and the detection distance is 0.3 to 6 meters) and the goal's relative polar coordinates $[d^t, \theta^t]$. Inspired by [12] and LSTM [13]/GRU [14], we preprocess the raw lidar data into a local obstacle map (LOMap), which we then input into the network alongside maps from the previous three timesteps [15]. This method provides a compact scene representation, enhancing the robot's ability to differentiate between similar scenes and improving its historical memory for escaping local minima aera.

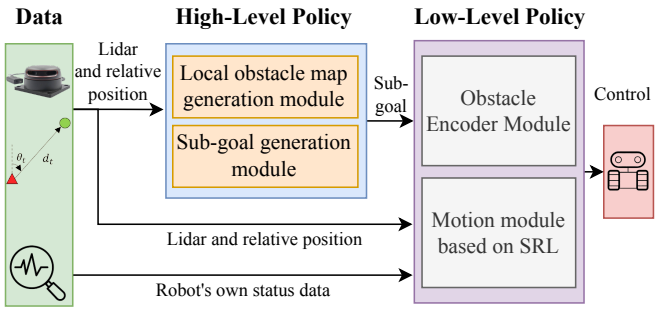
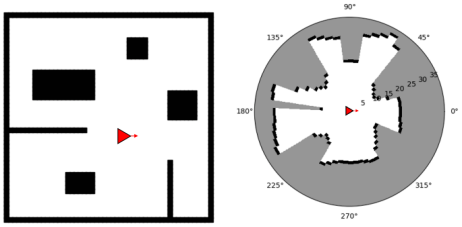


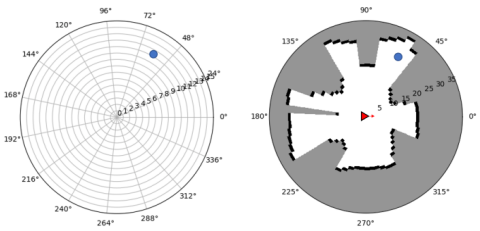
Fig. 3. Overall of the HRL-based safe mapless navigation policy.



(a) Environment (b) Local obstacle map

Fig. 4. An illustration of local obstacle map. The red triangle denotes the robot, with its arrow indicating the robot's direction. The unoccupied areas are depicted in white, the occupied areas in black, and the unexplored areas in gray.

2) Action Space: As shown in Fig. 5, the action space of the high-level policy consists of the polar coordinates of the sub-goal relative to the current position of the robot, denoted as $g_{s,i} = [d_{sg,i}, \theta_{sg,i}]$, where $d_{sg,i}$ is in meters and $\theta_{sg,i}$ is in radians. The sub-goal coordinates are constrained to the unoccupied areas within the generated local obstacle map. When the sub-goal output by the policy network are in areas that the robot cannot reach, we use a mask operation.



(a) (b)

Fig. 5. The polar coordinate of the sub-goal. We discretize the detectable range of the robot's lidar into 15 equal distance intervals and 360° into 15 equal angles, thereby discretizing the local obstacle map into 225 sector-shaped areas. The blue dot represents a sub-goal.

Authorized licensed use limited to: HUNAN UNIVERSITY. Downloaded on October 21,2026 at 09:12:10 UTC from IEEE Xplore. Restrictions apply.

3) Reward Function: To encourage the robot to reach its goal promptly, the reward function is designed as follows:

$$r = r_{\text{arrival}} + r_{\text{step}} + r_{\text{dist}} + r_{\text{out}}\,, \tag{1}$$

where r_{arrival} represents the reward for reaching the goal, which can be defined as:

$$r_{\text{arrival}} = \begin{cases} 100, d_t < d_{\text{limit}} \\ 0, \quad \text{otherwise} \end{cases}, \tag{2}$$

when the robot's distance to the goal d_t is less than the threshold d_{limit} (0.5 meters in this paper), the robot is considered to have reached the goal, and the reward r_{arrival} is 100; otherwise, r_{arrival} is 0. The value r_{step}= -1 indicates a penalty of -1 for each sub-goal selection, encouraging the robot to minimize detours and reach the goal promptly. The reward r_{dist} encourages the robot to approach the goal, calculated as the difference between consecutive timesteps using the A* algorithm on a 0.5×0.5 grid map:[1]

$$r_{dist} = \mu \left(d_{A*}^{t-1} - d_{A*}^{t} \right), \tag{3}$$

where the coefficient $\mu = 1$ in this paper. The penalty reward r_{out} = -20 applies when the robot selects an undesirable sub-goal, like one near or blocked by an obstacle. Though the A* algorithm slows training, it reduces robot wandering

4) Policy Network: The architecture of the high-level policy network are shown in Fig. 6. The reinforcement learning algorithm known as dueling double deep Q-networks algorithm [16], [17] is utilized to train the high-level policy network. Given the sparse reward nature of robot navigation, the weight ratio of rewards can impact both navigation performance and training efficiency, despite reward reshaping. Additionally, the complexity and variability of indoor environments pose training challenges. Thus, we employ the hindsight experience replay [18] to refine the training.

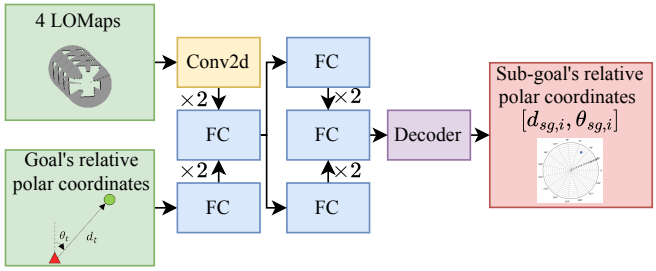


Fig. 6. An illustration of high-level policy network. 'Cond2v' for 2D convolutional layers and 'FC' for fully connected layers.

5) Environment Congestion-Based Sub-Goal Update: Since path planning requires a certain amount of calculation time, the robot significantly pauses when it reaches a sub-goal and then calculates the next one, resulting in low navigation efficiency. As the robot approaches the current sub-goal, we can generate the subsequent sub-goal, thereby alleviating the aforementioned phenomenon to a certain extent. However, in crowded scenes, with rapid changes in environment observations, setting the sub-goal update distance threshold too large can cause the robot to freeze or wander aimlessly. To address these issues, this paper proposes a dynamic update of sub-goals based on environment congestion. First, we define environment congestion at t as:

$$C^t = \frac{1}{n} \sum_{i=1}^{n} \frac{1}{\log_{d_s}(l_i^t + 1)}, \tag{4}$$

where n is the dimension of the lidar point cloud data $\mathbb{L}^t = [l_1^t, l_2^t, l_3^t, \ldots, l_n^t]$, which is 1080. d_s is the safe distance threshold, set to 3 meters. The larger the value of the environment congestion coefficient C^t, the more crowded the environment is deemed. If the environment congestion degree $C^t < 1$, it indicates that the environment is relatively unoccupied. Next, we define the sub-goal update distance threshold d_u^t. The update is triggered when the distance between the robot and the current sub-goal is less than d_u^t:

$$d_u^t = \text{clip}(\alpha C^t + \beta, 0.5, 2), \tag{5}$$

where α is a coefficient related to the robot's speed and maneuverability, taken as 0.25 in this paper, and β is also taken as 0.25. Furthermore, to prevent the robot from lingering in one area, a time threshold for sub-goal update is established at 30 seconds. That is, if the robot does not move within the sub-goal update distance threshold d_u^t within 30 seconds, a new sub-goal is forcibly regenerated. The sub-goal update mechanism, based on environment congestion, enables the robot to dynamically update sub-goals in response to surrounding obstacles and pedestrians, effectively preventing the robot from falling into local minimum areas.

C. Low-Level Sub-Goal-Conditioned Motion Planning Policy

The low-level motion policy is primarily tasked with enabling the robot to avoid both static and dynamic obstacles and must generate control instructions in real time [15]. As depicted in Fig. 3, this policy primarily comprises an obstacle encoder module for preprocessing state information and a motion planning module that utilizes SRL.

1) Observation Space: For obstacle avoidance during the robot's movement to the sub-goal, the observation space of the low-level motion planning policy encompasses the lidar point cloud data $\mathbb{L}^t$, the robot's movement speed $[v^t, \omega^t]$, and the polar coordinates of the sub-goal $g_{s,i} = [d_{sg,i}, \theta_{sg,i}]$) provided by the high-level policy. The current wheel odometer information is translated into the linear velocity v_t and angular velocity ω_t of robot as part of the observation space.

2) Action Space: The action space consists of the speed control commands for the robot, denoted as $[v^{t+1}, \omega^{t+1}]$, where the linear velocity v^t is measured in meters per second and the angular velocity ω^t is measured in radians per second.

3) Reward Function: We define the reward function as:

$$r = r_{\text{arrival}} + r_{\text{step}} + r_{\text{dist}}\,, \tag{6}$$

where each part is consistent with (1). Unlike typical reward functions, (6) omits the collision penalty term because SRL incorporates collision avoidance into its safety constraints.

[1]During training, the high-level policy network uses A*-based distances, eliminating the need for A* distance calculations during actual policy deployment, thus satisfying mapless navigation requirements.

Authorized licensed use limited to: HUNAN UNIVERSITY. Downloaded on October 21,2026 at 09:12:10 UTC from IEEE Xplore. Restrictions apply.

4) Policy Network and Training: The policy actor network architecture is depicted in Fig. 7. For the lidar point cloud data $\mathbb{L}^t$, we initially perform minimum pooling downsampling, followed by encoding obstacles within the safety distance threshold d_s to extract the adjacent obstacle set O^t_{list} (further details are provided in section III-C.5). The processed lidar data, combined with the sub-goal and the robot's current velocity data, forms the observation space code $\mathcal{H}_t$. Since the robot cannot obtain global information in a mapless navigation, it may end up in a wrong room. In order to solve this problem, $\mathcal{H}_t$ is concatenated with the $\mathcal{H}_{t-1}, \mathcal{H}_{t-2}, \mathcal{H}_{t-3}$ from the previous three time steps, enabling the robot to have historical memory functions. Ultimately, all observation space encoder information are fed into a three-layer fully connected network to determine $[v^t, \omega^t]$. Considering the safety of robot navigation, we use the SRL algorithm, constrained policy optimization (CPO) [19], to train the policy network. SRL streamlines reward design and balances the agent's behavior between conservative and aggressive approaches.

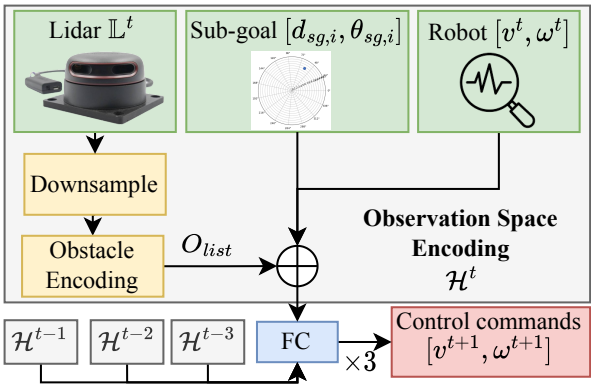


Fig. 7. Low-level motion planning Policy network.

5) Obstacle Encoding: We assume that the closer the obstacle is to the robot, the greater the threat to its navigation, and the closer the obstacle is to the robot's direction of movement, the more dangerous it is. In order to help the robot better perceive obstacles, this paper encodes them according to the threat level of the obstacles. For the 1080-dimensional single-line lidar point cloud data $\mathbb{L}^t$, we first remove the noise, and then take the frame with the smallest sampling distance value in every 30 frames of radar data points as the obstacle point cloud data at that angle, and then we get a 36-dimensional sparse radar data. Next, we encode the obstacle point cloud within the safety distance threshold d_s in the format of $\left[d_i^{o,t}, \theta_i^{o,t}\right]$ to obtain the adjacent obstacle set O^t_{list}, where $d_i^{o,t}$, $\theta_i^{o,t}$ represent the distance value of the ith obstacle point cloud and the angle of deviation from the positive direction of the robot, respectively. Finally, we sort the elements in O^t_{list} according to the threat level of the obstacles from small to large, and obtain:

$$O^t_{\text{list}} = \left(\left[d_0^{o,t}, \theta_0^{o,t}\right], \cdots, \left[d_i^{o,t}, \theta_i^{o,t}\right], \cdots, \left[d_n^{o,t}, \theta_n^{o,t}\right]\right), \tag{7}$$

where $n = 36$, for $i < j$, $d_i^{o,t} > d_j^{o,t}$ or $d_i^{o,t} = d_j^{o,t}, \theta_i^{o,t} \geq \theta_j^{o,t}$.

IV. Simulation Experiments

In this section, we present the results of our simulation experiments with the HRL-based navigation framework.

A. Experiment Settings

Due to the independence of the high-level and low-level policies, we train the two policy networks separately before deploying them in conjunction. The high-level sub-goal generation policy is initially trained and tested within a 2D Python simulator. Subsequently, for the low-level motion planning policy, we construct a simulation environment utilizing ROS-Kinetic Gazebo 9, wherein the robot model employed is the Turtlebot. To introduce dynamic obstacles into the simulation, we incorporate the social force model-based pedestrians [20].

B. Metrics

We designed metrics to evaluate navigation performance:

1) Success rate (SR): The ratio of successful goal reaches to total tests. We conduct 100 tests in each scenario.

2) Success Rate of Non-collision (SRN): The ratio of successful, collision-free goal reaches to total tests.

3) Average Collision Times (CT): The average number of collisions per test. Rounds aren't terminated by collisions, so the CT can exceed 1.

4) Success weighted by Path Length (SPL) [21]: A measure of navigation path quality, calculated as:

$$SPL = \frac{1}{N}\sum_{i=1}^{N} S_i \frac{\tilde{p}_i}{\max\left(p_i, \tilde{p}_i\right)}, \tag{8}$$

where S_i is 1 for a successful and 0 for an unsuccessful navigation. $\tilde{p}_i$ is the theoretical shortest path, which is calculated by A* algorithm. p_i is the actual path length, and N is the number of trials. Higher SPL values indicate better navigation.

5) Success weighted by Navigation Time (SNT): A new metric assessing time efficiency, defined as:

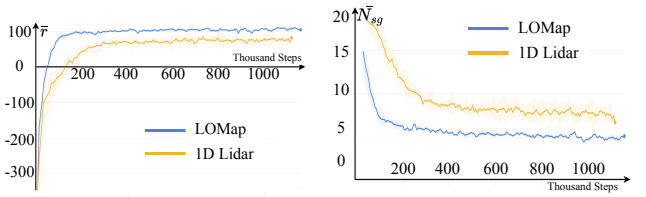
$$SNT = \frac{1}{N}\sum_{i=1}^{N} S_i \frac{t_i}{\max\left(q_i, t_i\right)}, \tag{9}$$

where q_i is the actual navigation time, t_i is the shortest time for the ith navigation, obtained by dividing the optimal distance based on A* algorithm by the robot?s maximum speed.

C. Ablation Experiments

1) LOMap for Sub-Goal Generation: To assess the enhancement in navigation performance afforded by the incorporation of LOMaps, we compare the use of raw lidar data with that of LOMaps as inputs for lidar information. Fig. 8(a) and 8(b) demonstrate that the approach utilizing LOMaps as input achieves a higher average reward more rapidly and reaches the goal with fewer sub-goals. In conclusion, incorporating LOMaps as state inputs allows the robot to develop a more effective policy, facilitating long-term planning and making it particularly advantageous for sub-goal generation.

Authorized licensed use limited to: HUNAN UNIVERSITY. Downloaded on October 21,2026 at 09:12:10 UTC from IEEE Xplore. Restrictions apply.



(a) Average reward $\bar{r}$ (b) Average number of sub-goals $\bar{N}_{sg}$

Fig. 8. Training curves of Lidar data and LOMap in high-level policy.

2) Reward Function for Sub-Goal Generation: For the reward r_{dist} defined in (3), we employ distance guidance based on the A* algorithm. This approach is contrasted with guidance based on Euclidean distance and a sparse reward scheme, where the reward is set to $r = 0$. As depicted in Fig. 9, utilizing A*-based distance guidance facilitates the robot's acquisition of skills like navigating through doorways and around obstacles, prevents entrapment in local minima area, and allows for goal attainment in a smaller number of sub-goals.

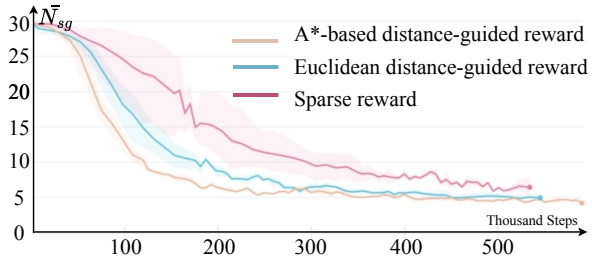


Fig. 9. The average number of sub-goals under different guidance reward.

3) Scenarios for for Sub-Goal Generation: We assess the high-level sub-goal generation policy based on navigation distance, the presence of local minimum areas, and other conditions.

4) Dynamic Obstacle Density for Low-level Policy: We assess the performance of the low-level motion planning policy by varying the density of dynamic obstacles. As evidenced in Table I, the policy, which is trained by SRL, facilitates the robot's navigation through dynamic environments with a sparse population. However, it encounters challenges in scenarios characterized by dense pedestrian traffic, occasionally resulting in collisions. This is primarily attributed to the robot's tendency to attempt navigating through the midst of groups of people moving in concert. Consequently, to mitigate the risk of unnecessary collisions, we incorporate post-processing techniques during the physical deployment of the navigation policy. Specifically, we implement a mechanism to preemptively suppress speed commands that lead to collisions with pedestrians.

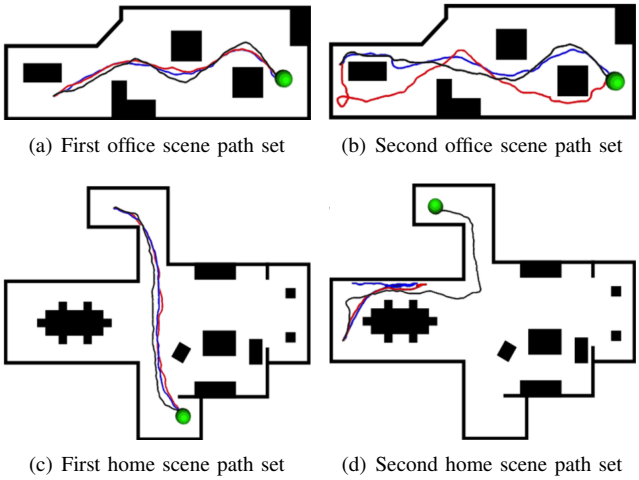
D. Comparison Experiments

1) Static Obstacle Scene: We first simulate robot delivery or food delivery in static environments in office and home scenarios. The static scene navigation performance test is mainly to evaluate the mobile robot's ability to escape from local minima area and drive in a straight line. We use the low-level policy based on proximal policy optimization (PPO) [22] and CPO and the navigation policy based on HRL to test the navigation performance.

TABLE I

RESULTS OF LOW-LEVEL POLICY IN DYNAMIC ENVIRONMENTS.

Number of Pedestrians	SR	SRN	CT	SPL	SNT
0	100	100	0	93.62	90.1
5	100	100	0	91.05	90.17
10	100	98	2	85.78	77.73
15	100	96	7	83.25	72.21



(a) First office scene path set (b) Second office scene path set

(c) First home scene path set (d) Second home scene path set

Fig. 10. Simulation results of paths in the static scenarios. The red, blue, and black curves represent the robot paths obtained from the low-level policies using the PPO and CPO algorithms, and the HRL-based navigation policy, respectively.

TABLE II

SIMULATION PATH RESULTS IN STATIC SCENES.

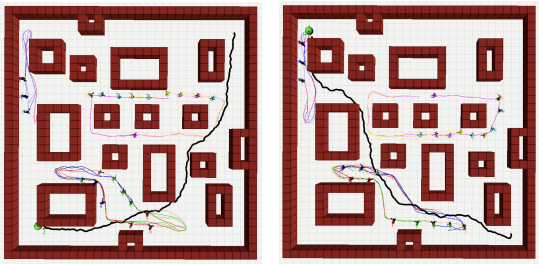
Scenerio	Index	Policy	Metrics				
			SR	SRN	CT	SPL	SNT
Office	1st	Low-Level (PPO)	100	93	1.27	87.49	80.91
		Low-Level (CPO)	100	100	0	92.26	85.12
		HRL	**100**	**100**	**0**	**92.56**	**87.66**
	2nd	Low-Level (PPO)	95	0	39.30	73.52	59.95
		Low-Level (CPO)	98	42	15.34	86.59	69.75
		HRL	**100**	**83**	**3.37**	**91.77**	**73.19**
Home	1st	Low-Level (PPO)	100	85	0.32	**92.33**	**90.57**
		Low-Level (CPO)	100	100	0	90.52	83.23
		HRL	**100**	**100**	**0**	90.87	84. 97
	2nd	Low-Level (PPO)	0	0	63.15	0	0
		Low-Level (CPO)	0	0	23.72	0	0
		HRL	**97**	**92**	**0.43**	**91.16**	**84.95**

As shown in Table II, in simple office and home scenarios like those in Fig. 10(a) and Fig. 10(c), the three navigation policies perform similarly, except that the PPO-based low-level policy has a higher collision probability when navigating corners. In scenarios with narrow channels, such as those in Fig. 10(b) and Fig. 10(d), the performance of all three policies is significantly reduced, resulting in tortuous paths and numerous collisions. The long path lengths and complex

Authorized licensed use limited to: HUNAN UNIVERSITY. Downloaded on October 21,2026 at 09:12:10 UTC from IEEE Xplore. Restrictions apply.

obstacles make it impossible for the PPO-based and CPO-based low-level policies to navigate to the goal. However, the HRL-based navigation policy excels over the PPO-based and CPO-based policies, particularly in the local minimum area depicted in Fig. 10(d). It can successfully escape and guide the robot to the goal. These results demonstrate that the HRL-based navigation policy possesses strong generalization capabilities, enabling efficient navigation in indoor scenes with local minimum areas that have not been seen during policy training.

2) Dynamic Obstacle Scene: The dynamic navigation experiment modeles the conditions of a robot in crowded environments like restaurants, primarily assessing the robot's obstacle avoidance and rapid response abilities within dense crowds. Concurrently, to evaluate the generalization performance of the HRL-based navigation policy, we employ the policy model trained in the static experiment without additional fine-tuning.



(a) (b)

Fig. 11. Robot path in the dynamic experiment (black). The green dot is the robot's goal, and the other colored dots represent pedestrians and their trajectories. The simulation environment models the layout of a restaurant, with a total of 23 dynamically simulated pedestrians.

TABLE III
SIMULATION RESULTS IN DYNAMIC SCENARIOS.

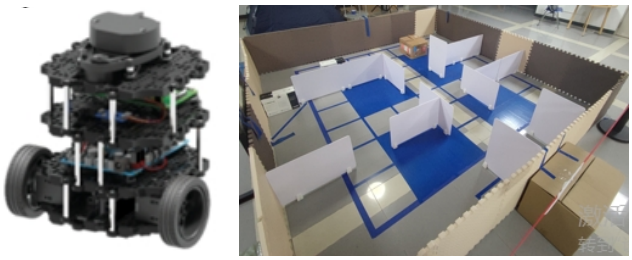
No.	Velocity of Pedestrians	SR	SRN	CT
1	0.5 m/s	100	100	0
2	0.8 m/s	97	90	0.15
3	1.2 m/s	95	78	0.75

* The number of pedestrians is 23.

Fig. 11 illustrates that the robot follows a nearly optimal path and timely avoids crowds. Table III shows that in dynamic scenes with slow pedestrians, the HRL-based navigation policy maintains a low collision rate for the robot without post-processing. However, as pedestrian speed increases, the robot's success rate in collision-free navigation gradually decreases, while its collision rate significantly rises.

V. Validation Experiments

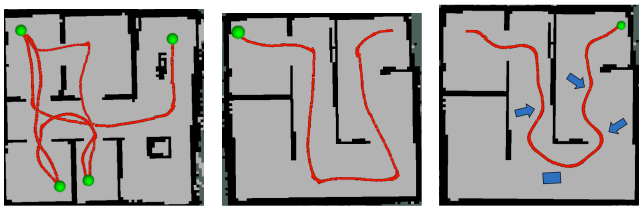
As shown in Fig. 12, this paper conducts physical experiments using the TurtleBot3-Burger robot, equipped with a single-line lidar. Based on the actual hardware parameters of the TurtleBot3, we set the maximum linear velocity output of the neural network to 0.22 m/s. The experimental scenes, which include static and dynamic obstacles, are constructed using plastic plates.



(a) TurtleBot3-Burger (b) Real-world navigation environment

Fig. 12. Physical robot and environment.

We initially conduct 30 navigation tests in the scenario depicted in Fig. 13(a) with four goals and 10 tests in the long-distance navigation scenario shown in Fig. 13(b). Despite hardware parameter variations, the policy model consistently guides the robot to successfully reach goals in both local minimum areas and long-distance navigation scenarios without aimless movement. Subsequently, we introduce dynamic obstacles into the scenario from Fig. 13(b) to assess dynamic navigation performance, with results presented in Fig. 13(c). The results indicate that TurtleBot3 can timely adjust its direction to avoid interfering obstacles, resulting in no collisions. These experiments demonstrate that the HRL-based navigation policy performs effectively on physical robots and exhibits strong generalization capabilities.



(a) (b) (c)

Fig. 13. TurtleBot3's navigation path in the physical experiment (red). The green dot indicates the robot's goal. (a) Static scene with four goals. (b) Static scene for long-distance navigation. (c) Dynamic Scene. Arrows indicate external dynamic interference.

VI. Conclusions

This paper presents a robust mapless navigation framework leveraging HRL to effectively navigate local minima area in indoor environments. The high-level policy's dynamic sub-goal updates, coupled with an enhanced obstacle encoding method at the low level, ensure real-time adaptability and improved environmental perception. Through detailed simulations across diverse settings, including offices, homes, and restaurants, our HRL framework has consistently demonstrated superior performance in navigating static and dynamic obstacles alike. The successful physical deployment on a TurtleBot3 robot further underscores the practical applicability and reliability of our navigation policy. While the HRL-based navigation policy performs well in various settings, it encounters occasional collisions in scenes with dense obstacles.

Authorized licensed use limited to: HUNAN UNIVERSITY. Downloaded on October 21,2026 at 09:12:10 UTC from IEEE Xplore. Restrictions apply.

REFERENCES

[1] A. Loganathan and N. S. Ahmad, "A systematic review on recent advances in autonomous mobile robot navigation," *Eng. Sci. Technol.*, vol. 40, p. 101343, 2023.

[2] L. Dong, Z. He, C. Song, and C. Sun, "A review of mobile robot motion planning methods: from classical motion planning workflows to reinforcement learning-based architectures," *J. Syst. Eng. Electron.*, vol. 34, no. 2, pp. 439–459, 2023.

[3] A. Patil, S. Chandolkar, P. Kothawade, S. Shaikh, R. Jaiswal, and A. Patwardhan, "A comprehensive overview of bug algorithms for decentralized mapping and navigation," in *Proc. Intell. Conf. Smart Comput. Commun.*, 2024, pp. 323–333.

[4] K. Yan, J. Gao, and Y. Li, "Deep reinforcement learning based mobile robot navigation using sensor fusion," in *Proc. Chinese Control Conf.*, 2023, pp. 4125–4130.

[5] M. Melchiorre, L. Salamina, L. S. Scimmi, S. Mauro, and S. Pastorelli, "Experiments on the artificial potential field with local attractors for mobile robot navigation," *Robot.*, vol. 12, no. 3, p. 81, 2023.

[6] Z. Liu, Y. Zhai, J. Li, G. Wang, Y. Miao, and H. Wang, "Graph relational reinforcement learning for mobile robot navigation in large-scale crowded environments," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 8, pp. 8776–8787, 2023.

[7] L. Brunke, M. Greeff, A. W. Hall, Z. Yuan, S. Zhou, J. Panerati, and A. P. Schoellig, "Safe learning in robotics: From learning-based control to safe reinforcement learning," *Annu. Rev. Control Robot Auton. Syst.*, vol. 5, no. 1, pp. 411–444, 2022.

[8] Y. Jang, J. Baek, and S. Han, "Hindsight intermediate targets for mapless navigation with deep reinforcement learning," *IEEE Trans. Ind. Electron.*, vol. 69, no. 11, pp. 11 816–11 825, 2022.

[9] C. Gebauer, N. Dengler, and M. Bennewitz, "Sensor-based navigation using hierarchical reinforcement learning," in *Proc. Int. Conf. Intell. Auton. Syst.* Springer, 2022, pp. 546–560.

[10] J. De Heuvel, W. Shi, X. Zeng, and M. Bennewitz, "Subgoal-driven navigation in dynamic environments using attention-based deep reinforcement learning," in *Proc. Int. Conf. Adv. Rob.*, 2023, pp. 79–85.

[11] Y. Gao, J. Wu, X. Yang, and Z. Ji, "Efficient hierarchical reinforcement learning for mapless navigation with predictive neighbouring space scoring," *IEEE Trans. Autom. Sci. Eng.*, vol. 21, no. 4, pp. 5457–5472, 2024.

[12] C. Gebauer and M. Bennewitz, "The pitfall of more powerful autoencoders in lidar-based navigation," *arXiv preprint arXiv:2102.02127*, 2021.

[13] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, 1997.

[14] K. Cho, B. Van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, "Learning phrase representations using rnn encoder-decoder for statistical machine translation," *arXiv preprint arXiv:1406.1078*, 2014.

[15] X. Pang, Y. Li, Q. Liu, and K. Deng, "A mapless navigation method based on reinforcement learning and local obstacle map," in *Proc. China Autom. Congr.*, 2022, pp. 5754–5759.

[16] Z. Wang, T. Schaul, M. Hessel, H. Hasselt, M. Lanctot, and N. Freitas, "Dueling network architectures for deep reinforcement learning," in *Proc. Intell. Conf. Mach. Learn.*, 2016, pp. 1995–2003.

[17] H. Van Hasselt, A. Guez, and D. Silver, "Deep reinforcement learning with double q-learning," in *Proc. AAAI Conf. Artif. Intell. Interact. Digit. Entertain.*, 2016, pp. 2094–2100.

[18] M. Andrychowicz, F. Wolski, A. Ray, J. Schneider, R. Fong, P. Welinder, B. McGrew, J. Tobin, O. Pieter Abbeel, and W. Zaremba, "Hindsight experience replay," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, pp. 5055–5065.

[19] J. Achiam, D. Held, A. Tamar, and P. Abbeel, "Constrained policy optimization," in *Proc. Int. Conf. Mach. Learn.*, 2017, pp. 22–31.

[20] D. Helbing and P. Molnar, "Social force model for pedestrian dynamics," *Physical review E*, vol. 51, no. 5, p. 4282, 1995.

[21] P. Anderson, A. Chang, D. S. Chaplot, A. Dosovitskiy, S. Gupta, V. Koltun, J. Kosecka, J. Malik, R. Mottaghi, M. Savva *et al.*, "On evaluation of embodied navigation agents," *arXiv preprint arXiv:1807.06757*, 2018.

[22] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.

Authorized licensed use limited to: HUNAN UNIVERSITY. Downloaded on October 21,2026 at 09:12:10 UTC from IEEE Xplore. Restrictions apply.